

Dynamic Programming

Algorithm Design





团体作业： 3-5人一组，算法问题自选，围绕算法展开陈述，需准备Ppt，分析问题、代码呈现（可选伪码）、可视化等内容，总时长不超过15min。11周确定分组，15周开始分组展示。

个人作业： 自行选择一个算法问题，请以学术论文规范形式完成，报告以计算机学报模板完成，字数**1500+**，具体要求如下：

- 1.报告包括题目（自拟），摘要，引言，其中引言部分，需介绍问题的研究背景和研究意义，而不是脱离实际意义的单个实例；
- 2.对算法进行分析描述，比如给出算法的递归方程，诸如此类；
- 3.分析算法计算复杂度；
- 4.给出算法伪代码；
- 5.实验结果（运用你熟悉的代码，推荐以 Matlab、Python 给出数值算例，实验结果应尽量丰富，如果有代码，请放在附录）；
- 6.结论。



西南财经大学
SOUTHWESTERN UNIVERSITY OF FINANCE AND ECONOMICS

Algorithm Design & Analysis

Lecture 7

西南财经大学

经世济民，孜孜以求

动态规划算法的设计与分析

投资问题

背包问题

最长公共子
序列问题

递归实现

迭代实现

动态规划的设计要素
矩阵链相乘

动态规划的设计思想
以最短路径算法为例介绍设计思想及相关条件





一、动态规划算法的例子

最短路径算法

二、动态规划的设计要素

递归实现

迭代实现

三、动态规划实例

投资问题

背包问题

最长公共子序列





● 最短路径

● 问题定义

● SDP实例

● 算法设计

● 实例分析

● 子问题界定

● 依赖关系

● 优化原则

● 反例

● 小结

最短路径问题

输入： 起点集合 $\{S_1, S_2, \dots, S_n\}$

终点集合 $\{T_1, T_2, \dots, T_m\}$

中间结点集,

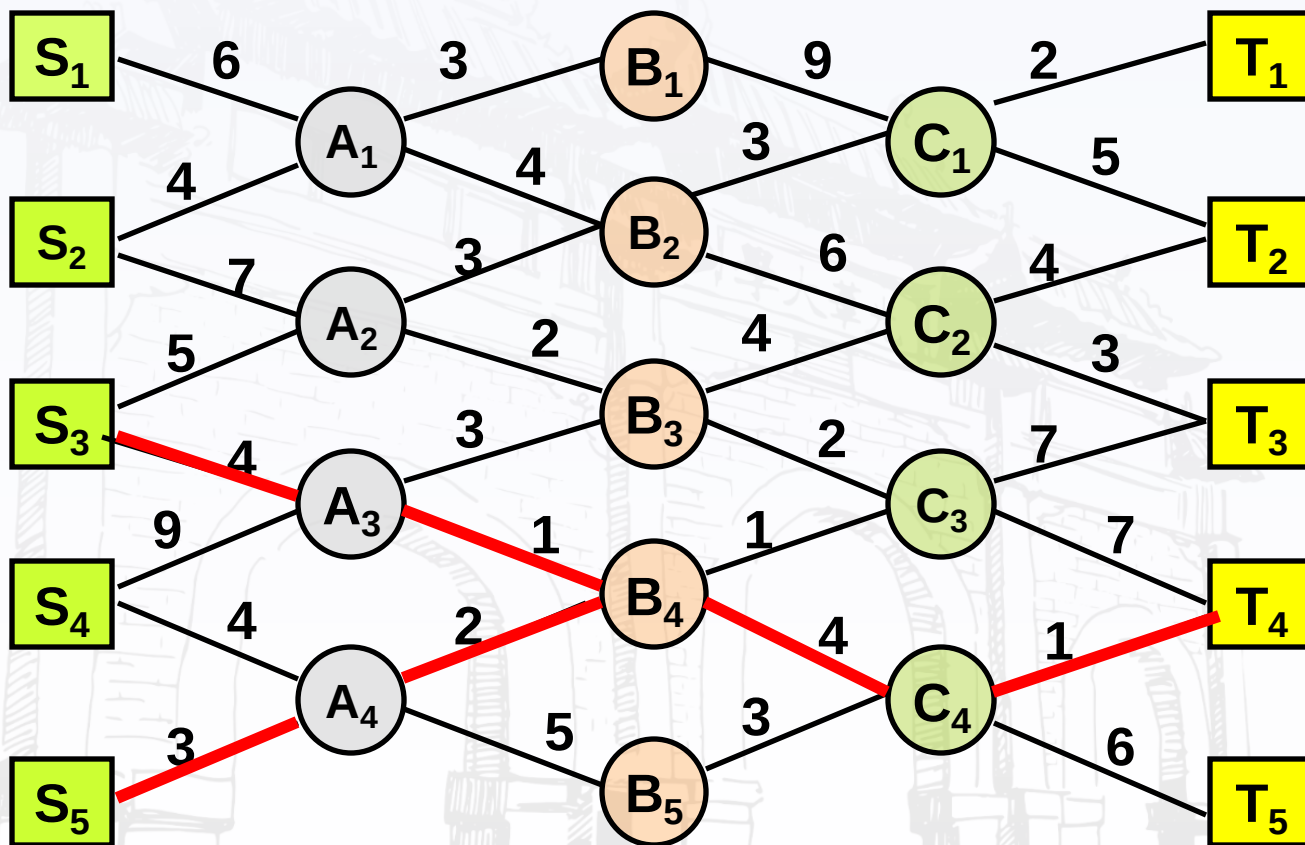
边集 E , 对于任意边 e 有长度

输出： 一条从起点到终点的最短路径



最短路径

最短路径实例





● 最短路径

● 问题定义

● SDP实例

● 算法设计

● 实例分析

● 子问题界定

● 依赖关系

● 优化原则

● 反例

● 小结

算法设计

蛮力算法：考察每一条从某个起点到某个终点的路径，计算长度，从其中找出最短路径。

此处蛮力算法有多少条路径？

在上述实例中，如果网络的层数为 k ，那么路径条数将接近 2^k

动态规划算法：多阶段决策过程.每步求解的问题是后面阶段求解问题的子问题.每步骤决策将依赖于以前步骤的决策过程.

● 最短路径

● 问题定义

● SDP实例

● 算法设计

● 实例分析

● 子问题界定

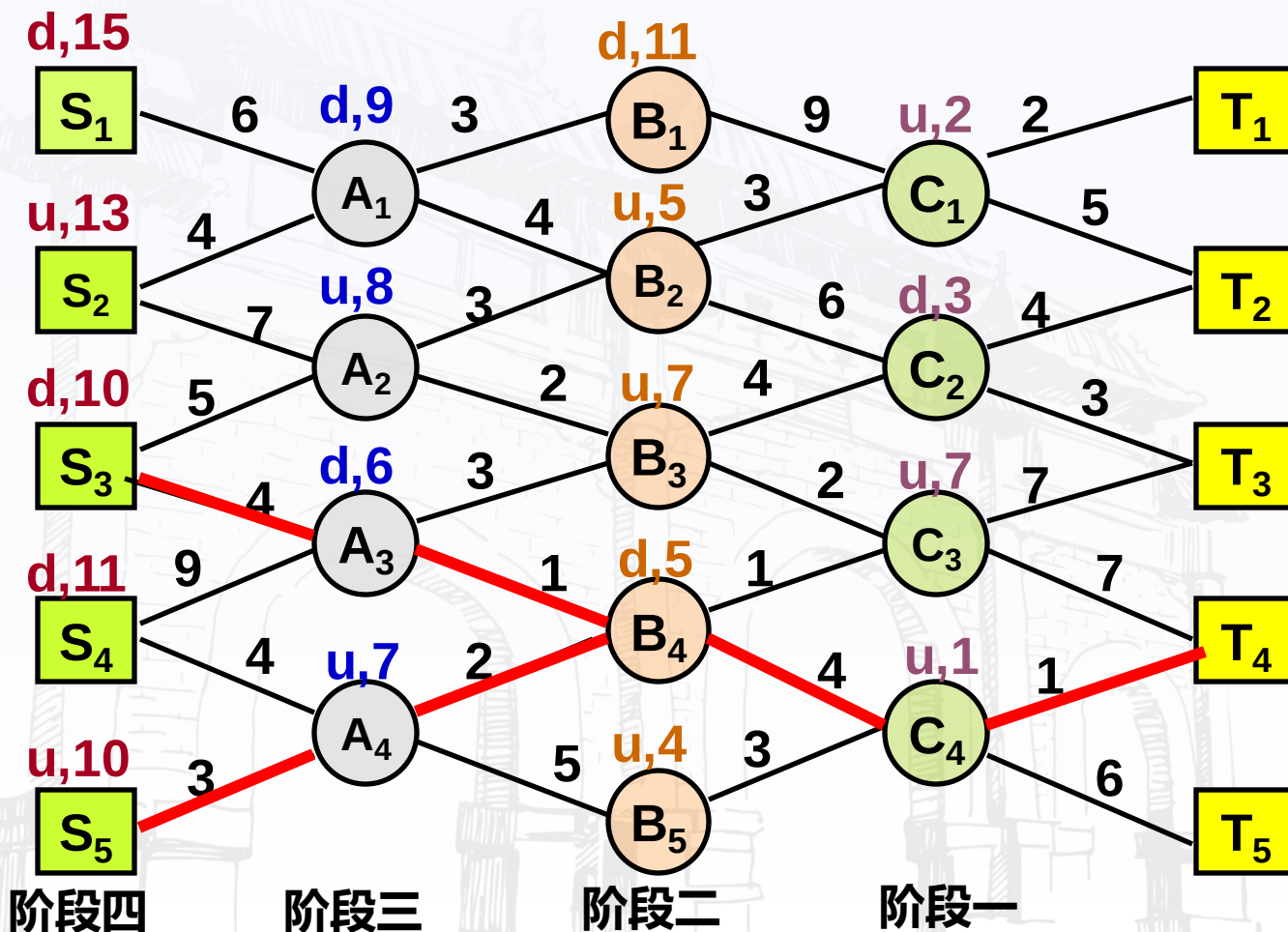
● 依赖关系

● 优化原则

● 反例

● 小结

最短路径 (SDP) 实例





● 最短路径

● 问题定义

● SDP实例

● 算法设计

● 实例分析

● 子问题界定

● 依赖关系

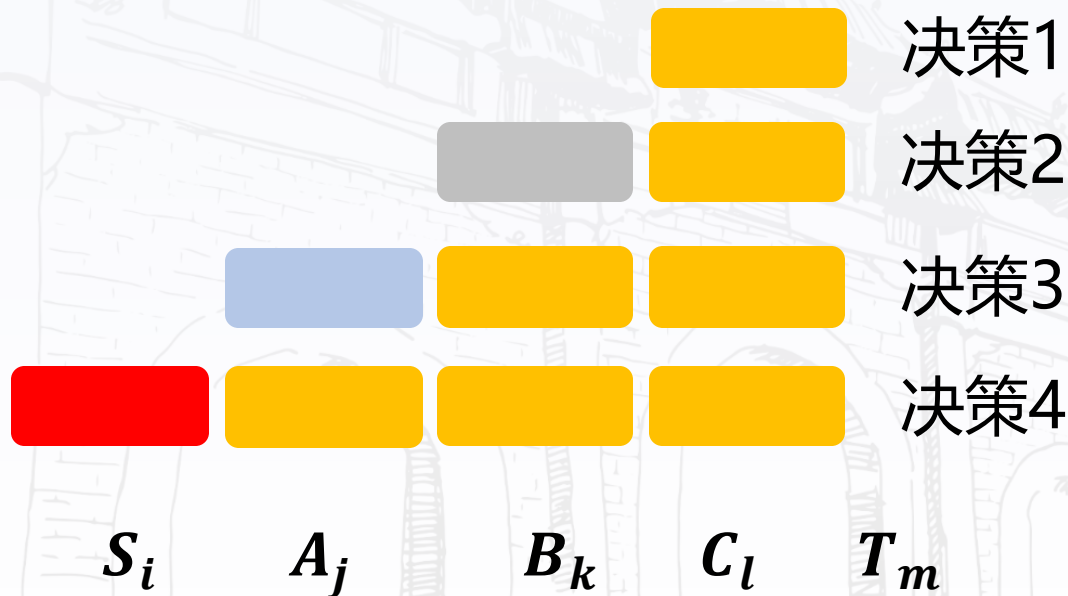
● 优化原则

● 反例

● 小结

SDP需要考虑哪些---子问题界定

后边界不变，前边界前移





● 最短路径

● 问题定义

● SDP实例

● 算法设计

● 实例分析

● 子问题界定

● 依赖关系

● 优化原则

● 反例

● 小结

SDP的依赖关系

$$\underline{F(C_l)} = \min_m \{C_l T_m\}$$

决策1

$$F(B_k) = \min_l \{B_k C_l + \underline{F(C_l)}\}$$

决策2

$$F(A_j) = \min_k \{A_j B_k + F(B_k)\}$$

决策3

$$F(S_i) = \min_j \{S_i A_j + F(A_j)\}$$

决策4

优化函数之间存在依赖关系，也即动态规划的优化原则



● 最短路径

● 问题定义

● SDP实例

● 算法设计

● 实例分析

● 子问题界定

● 依赖关系

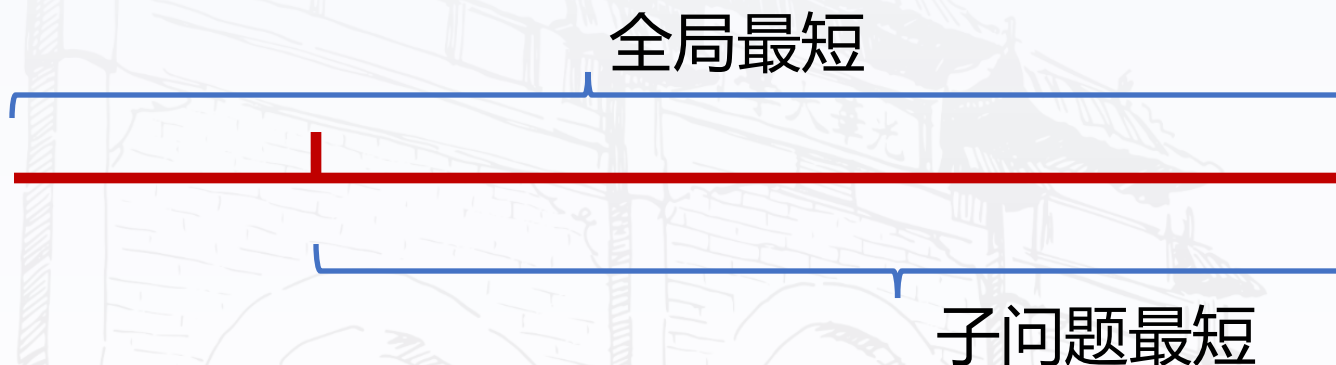
● 优化原则

● 反例

● 小结

优化原则:最优子结构性质

优化函数的特点: 任何最短路径的子路径相对于子问题始点、终点也最短.



优化原则: 一个最优决策序列的任何子序列本身一定是相对于子序列的初始状态和结束状态的最优决策序列.

思考: 局部最优才能保证全局最优, 一定可行么?

● 最短路径

● 问题定义

● SDP实例

● 算法设计

● 实例分析

● 子问题界定

● 依赖关系

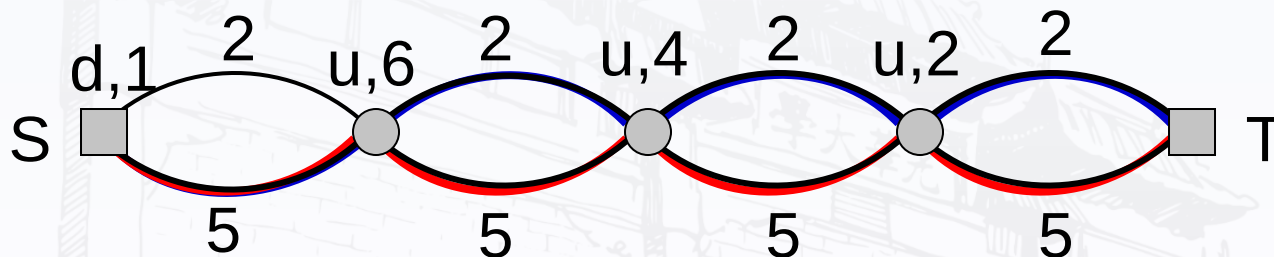
● 优化原则

● 反例

● 小结

优化原则:一个反例 局部最优都是全局最优才可行

求总长模10的最小路径



最优解: 下、下、下、下

动态规划算法的解: 下、上、上、上

不满足优化原则, 不能使用动态规划设计技术

破坏了依赖关系, 子问题的解与全局最优解不一致

小结

动态规划 (Dynamic Programming)

- 求解过程是**多阶段决策过程**，**每步处理一个子问题**，可用于求解组合优化问题
- 适用条件：问题要满足**优化原则**或**最优子结构性**，即：一个最优决策序列的任何子序列本身**一定是**相对于子序列的初始和结束状态的**最优决策序列**
简而言之：局部最优都是全局最优才可行



一、动态规划算法的例子

最短路径算法

二、动态规划的设计要素

递归实现

迭代实现

三、动态规划实例

投资问题

背包问题

最长公共子序列





设计要素

- 矩阵链问题
- 基本运算次数
- 实例
- 实例分析
- 蛮力算法
- 动态规划算法
- 递推方程
- 小结

动态规划设计要素

1. 问题建模，优化**目标函数**是什么？**约束条件**是什么？
2. 如何**划分子问题**，**参数**如何界定（边界）？
3. **原问题**的优化函数值与**子问题**的优化函数值存在着什么**依赖关系**？（递推方程）
4. 递推方程是否满足优化原则？不成立则不能使用
5. 最小子问题怎样界定？其优化函数值，即初值等于什么？

动态规划没有统一的模式，需要具体问题具体分析



设计要素

矩阵链问题

基本运算次数

实例

实例分析

蛮力算法

动态规划算法

递推方程

小结

矩阵链相乘

问题 设 $A_1, A_2, A_3, \dots, A_n$ 为矩阵序列,
 A_i 为 $P_{i-1} \times P_i$ 阶的矩阵, $i = 1, 2, \dots, n$. 试
确定矩阵乘法的顺序, 使得元素相乘总次数
最少.

输入: 向量,

$$P = \langle P_0, P_1, \dots, P_n \rangle,$$

其中 $P_0, P_1, \dots, P_n$ 为 n 个矩阵的行数与列数

输出: 矩阵链乘法加括号的位置

设计要素

- 矩阵链问题
- 基本运算次数
- 实例
- 实例分析
- 蛮力算法
- 动态规划算法
- 递推方程
- 小结

矩阵相乘基本运算次数

矩阵A: i 行 j 列, **B**: j 行 k 列, 以元素相乘作基本运算, 计算 AB 的工作量

$$\begin{bmatrix} \dots & \dots & \dots \\ a_{t1} & a_{t2} & \dots a_{tj} \\ \dots & \dots & \dots \end{bmatrix} \begin{bmatrix} \vdots & \begin{bmatrix} b_{1s} \\ b_{2s} \\ \vdots \\ b_{js} \end{bmatrix} & \vdots \end{bmatrix} = \begin{bmatrix} \vdots & \begin{bmatrix} c_{ts} \end{bmatrix} & \vdots \end{bmatrix}$$

$$c_{ts} = a_{t1}b_{1s} + a_{t2}b_{2s} + \dots + a_{tj}b_{js}$$

AB : i 行 k 列, 计算每个元素需要做 j 次乘法, 总计乘法次数为 ijk



设计要素

- 矩阵链问题
- 基本运算次数
- 实例
- 实例分析
- 蛮力算法
- 动态规划算法
- 递推方程
- 小结

矩阵相乘实例

实例: $P = \langle 10, 100, 5, 50 \rangle$

$A_1: 10 \times 100, \quad A_2: 100 \times 5, \quad A_3: 5 \times 50,$

乘法次序?

$(A_1 A_2) A_3$: $10 \times 100 \times 5 + 10 \times 5 \times 50 = 7500$

$A_1(A_2 A_3)$: $10 \times 100 \times 50 + 100 \times 5 \times 50 = 75000$

第一种次序计算次数最少.



设计要素

- 矩阵链问题
- 基本运算次数
- 实例
- 实例分析
- 蛮力算法
- 动态规划算法
- 递推方程
- 小结

蛮力算法

加 n 个括号的方法有 $\frac{1}{n+1} \binom{2n}{n}$ 种,是一个Catalan数, 是指数级别

$$W(n) = \Omega\left(\frac{1}{n+1} \frac{(2n)!}{n! n!}\right)$$

代入
Stirling
公式

$$= \Omega\left(\frac{1}{n+1} \frac{\sqrt{2\pi 2n} \left(\frac{2n}{e}\right)^{2n}}{3\sqrt{2\pi n} \left(\frac{n}{e}\right)^n \sqrt{2\pi n} \left(\frac{n}{e}\right)^n}\right)$$

$$= \Omega(2^{2n} / n^{\frac{3}{2}})$$

复杂度极高, 蛮力算法不可行



设计要素

- 矩阵链问题
- 基本运算次数
- 实例
- 实例分析
- 蛮力算法
- 动态规划算法
- 递推方程
- 小结

动态规划算法

子问题划分

$A_{i..j}$: 矩阵链 $A_i A_{i+1} \dots A_j$, 界边 i, j

输入向量: $\langle P_{i-1}, P_i, \dots, P_j \rangle$

其最好划分的运算次数: $m[i, j]$

子问题的依赖关系

最优划分最后一次相乘发生在矩阵 k 的位置, 即

$$A_{i..j} = A_{i..k} A_{k+1..j}$$

$A_{i..j}$ 最优运算次数依赖于 $A_{i..k}$ 与 $A_{k+1..j}$ 的最优运算次数



设计要素

矩阵链问题

基本运算次数

实例

实例分析

蛮力算法

动态规划算法

递推方程

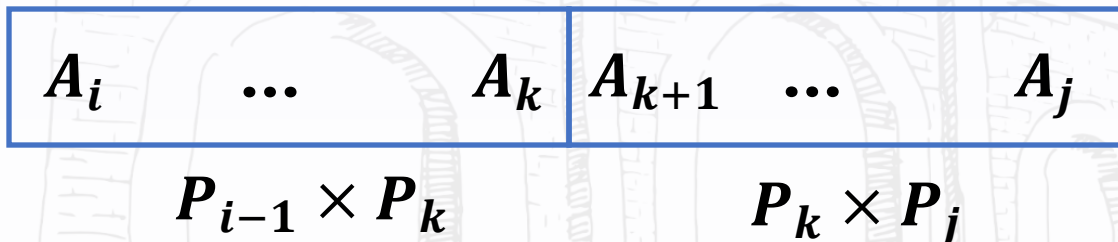
小结

优化函数的递推方程

递推方程：

$m[i, j]$ ：得到 $A_{i..j}$ 的最少的相乘次数

$$\underline{m[i, j]} = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{ \underline{m[i, k]} + \underline{m[k+1, j]} + P_{i-1}P_kP_j \} & i < j \end{cases}$$



该问题满足优化原则：**局部最优也能满足全局最优**

复杂度极高，需要遍历所有结果后才能获得最终结果



小结

动态规划算法设计要素

- 多阶段决策过程，**每步处理一个子问题**，用参数界定子问题的边界
- **列出**优化函数的**递推方程**及初值(**精髓**)
- 问题要满足优化原则或最优子结构性质，即：一个最优决策序列的任何子序列本身一定是相对于子序列的初值和结束状态的最优决策序列
局部最优也能满足全局最优



一、动态规划算法的例子

最短路径算法

二、动态规划的设计要素

递归实现

迭代实现

三、动态规划实例

投资问题

背包问题

最长公共子序列



● 递归实现

● 部分伪码

● 算法分析

● 时间复杂度

● 为何如此复杂

● 子问题分析

● 小结

矩阵链乘法部分伪码

算法 **RecurMatrixChain** (P, i, j)

子问题
和

1. $m[i, j] \leftarrow \infty$
2. $s[i, j] \leftarrow i$
3. for $k \leftarrow i$ to $j-1$ do
4. $q \leftarrow \text{RecurMatrixChain}(P, i, k)$
 $+ \text{RecurMatrixChain}(P, k+1, j) + p_{i-1}p_kp_j$
5. if $q < m[i, j]$
6. then $m[i, j] \leftarrow q$
7. $s[i, j] \leftarrow k$
8. return $m[i, j]$

划分k位
置

找到更好
的解

这里没有写出算法的全部描述（进入递归调用的初值等）



递归实现

部分伪码

算法分析

时间复杂度

为何如此复杂

子问题分析

小结

算法分析

时间复杂度的递推方程

$$T(n) \geq \begin{cases} O(1) & n = 1 \\ \sum_{k=1}^{n-1} (T(k) + T(n-k) + O(1)) & n > 1 \end{cases}$$

$$T(n) \geq O(n) + \sum_{k=1}^{n-1} T(k) + \sum_{k=1}^{n-1} T(n-k)$$

$$T(n) = O(n) + 2 \sum_{k=1}^{n-1} T(k)$$

其解仍然是一个指数函数



递归实现

部分伪码

算法分析

时间复杂度

为何如此复杂

子问题分析

小结

时间复杂度

证明: $T(n) \geq 2^{n-1}$

归纳法证明. 对于 $n > 1$, $T(n) = \Omega(2^{n-1})$

对于 $n = 2$, $T(2) \geq 2$

假设对于任何小于 n 的 k 命题为真, 则

$$T(n) = O(n) + 2 \sum_{k=1}^{n-1} T(k)$$

代入归纳
假设

$$\begin{aligned} &\geq O(n) + 2 \sum_{k=1}^{n-1} 2^{k-1} \\ &= O(n) + 2(2^{n-1} - 1) \geq 2^{n-1} \end{aligned}$$

等比数列
求和

蛮力算法 $\Omega(2^{2n}/n^{\frac{3}{2}})$

递归实现

部分伪码

算法分析

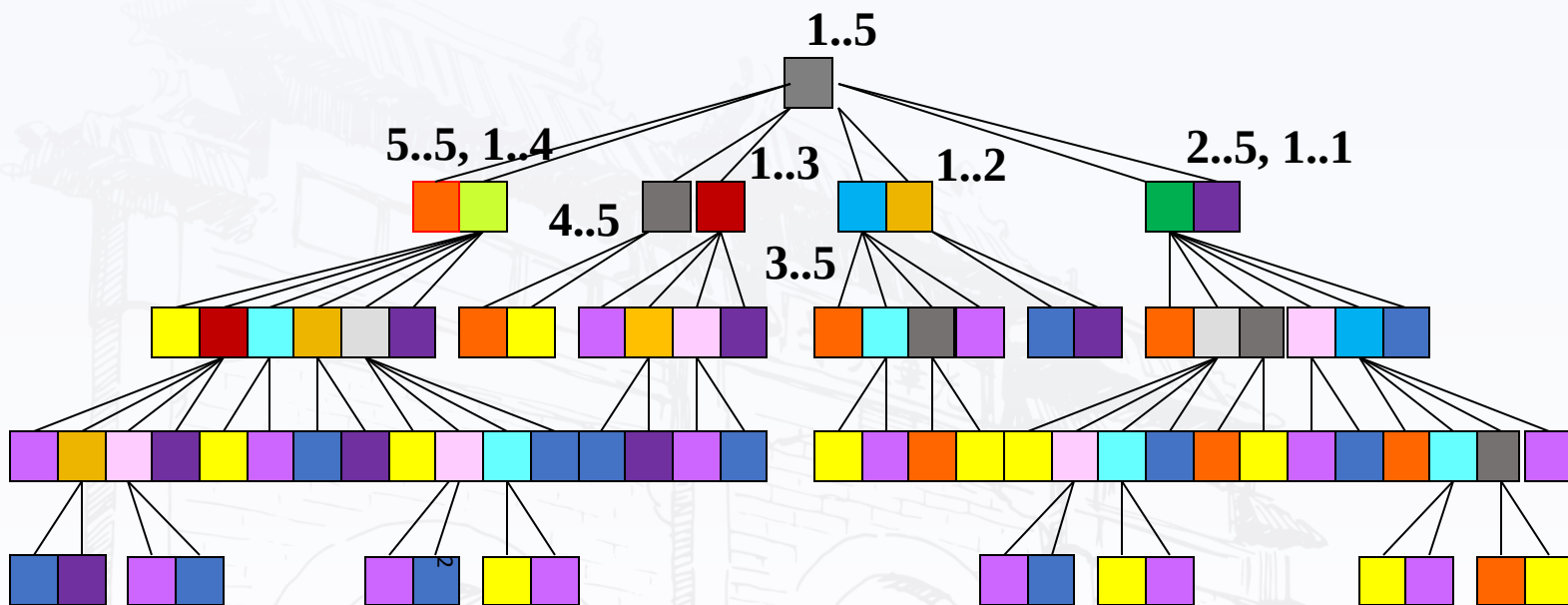
时间复杂度

为何如此复杂

子问题分析

小结

为什么复杂性高 --- 子问题重复计算



划分: $A_{1...4}A_{5...5}$, $A_{1...3}A_{4...5}$, $A_{1...2}A_{3...5}$, $A_{1...1}A_{2...5}$
产生8个子问题, 即第一层8个点



递归实现

部分伪码

算法分析

时间复杂度

为何如此复杂

子问题分析

小结

子问题分析

$n=5$, 计算子问题: 81个; 不同的子问题: 15个

子问题	1-1	2-2	3-3	4-4	5-5	1-2	2-3	3-4	4-5	1-3	2-4	3-5	1-4	2-5	1-5
数	8	12	14	12	8	4	5	5	4	2	2	2	1	1	1

边界不同的子问题15个, 递归计算的子问题81个

复杂度体现在子问题需要不断重复计算



小结

- 与蛮力算法相比，动态规划算法利用了子问题优化函数间的**依赖关系**，时间复杂度有所降低。
- 动态规划算法的递归实现效率不高，原因在于**同一子问题多次**重复出现，每次出现都需要重新计算一次。
- **采用空间换时间策略**，记录每个子问题首次计算结果，后面再用时就直接取值，每个子问题只计算一次。



一、动态规划算法的例子

最短路径算法

二、动态规划的设计要素

递归实现

迭代实现

三、动态规划实例

投资问题

背包问题

最长公共子序列





● 最短路径

● 计算关键

● 不同的子问题

● 迭代顺序

● 具体迭代例子

● 迭代实现

● 时间复杂度

● 实例

● 备忘录

● 标记函数

● 两种实现比较

● 小结

动态规划算法的迭代实现-计算关键

- 每个子问题只计算一次

- 迭代过程

- 从最小的子问题算起

- 考虑计算顺序，保证后面用到的值前面已经计算好

- 存储结构保存计算结果—备忘录

- 解的追踪

- 设计标记函数标记每步的决策

- 考虑根据标记函数追踪解的算法



● 最短路径

● 计算关键

● 不同的子问题

● 迭代顺序

● 具体迭代例子

● 迭代实现

● 时间复杂度

● 实例

● 备忘录

● 标记函数

● 两种实现比较

● 小结

矩阵链乘法不同子问题

长度1: 只含1个矩阵, 有 n 个子问题 (不需计算)

长度2: 含2个矩阵, $n - 1$ 个子问题

长度3: 含3个矩阵, $n - 2$ 个子问题

...

长度 $n - 1$: 含 $n - 1$ 个矩阵, 2个子问题

长度 n : 原始问题, 只有1个

分解为 n
个矩阵



● 最短路径

● 计算关键

● 不同的子问题

● 迭代顺序

● 具体迭代例子

● 迭代实现

● 时间复杂度

● 实例

● 备忘录

● 标记函数

● 两种实现比较

● 小结

矩阵链乘法迭代顺序

长度1: 初值, $m[i, j] = 0$

最开始的
计算次数

长度2: $1..2, 2..3, 3..4, \dots, n-1..n$

长度3: $1..3, 2..4, 3..5, \dots, n-2..n$

...

长度 $n-1$: $1..n-1, 2..n$

当计算链条长度为 $n-1$ 时, 有两个子问题1到 $n-1$, 2到 n

长度 n : $1..n$

当后续在计算时, 可直接调用前面的结果, 减少运算量



● 最短路径

● 计算关键

● 不同的子问题

● 迭代顺序

● 具体迭代例子

● 迭代实现

● 时间复杂度

● 实例

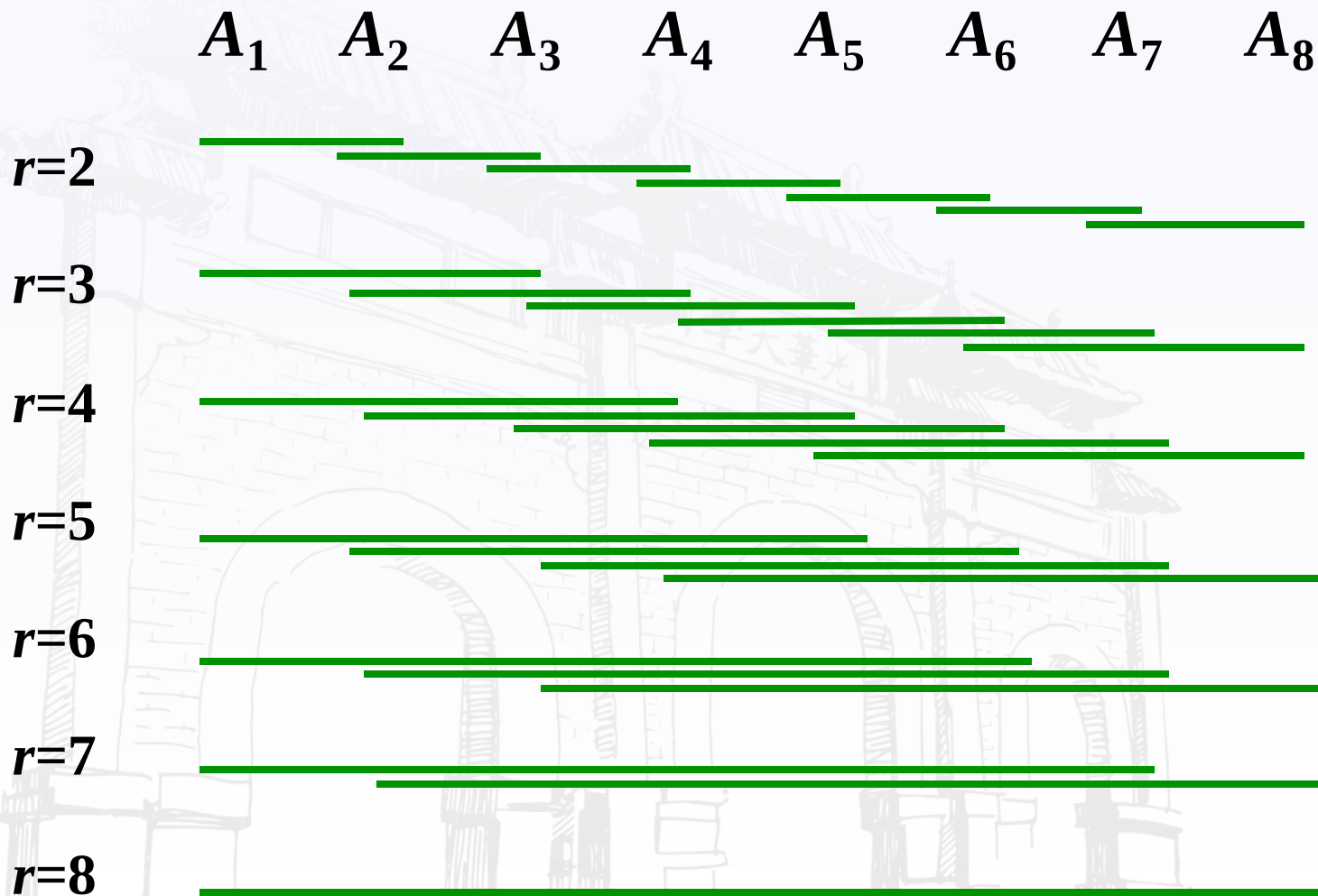
● 备忘录

● 标记函数

● 两种实现比较

● 小结

$n=8$ 的子问题计算顺序





● 最短路径

● 计算关键

● 不同的子问题

● 迭代顺序

● 具体迭代例子

● 迭代实现

● 时间复杂度

● 实例

● 备忘录

● 标记函数

● 两种实现比较

● 小结

算法：迭代实现

算法 MatrixChain(P, n)

1. 令所有的 $m[i, i]$ 初值为0 $1 \leq i \leq n$
2. for $r \leftarrow 2$ to n do // r 为计算的矩阵链长度
3. for $i \leftarrow 1$ to $n-r+1$ do // $n-r+1$ 为最后 r 链的左边界位置，遍历所有边界
4. $j \leftarrow i+r-1$ // 计算链 $i-j$ 右边界
5. $m[i, j] \leftarrow 0$ // $A_i(A_{i+1} \dots A_j)$
6. for $k \leftarrow i+1$ to $j-1$ do // 遍历所有 k 分割位置
7. $t \leftarrow m[i, k] + m[k+1, j] + p_{i-1}p_kp_j$ // $(A_i \dots A_k)(A_{k+1} \dots A_j)$
8. if $t < m[i, j]$
9. then $m[i, j] \leftarrow t$
10. $s[i, j] \leftarrow k$

二维数组 m 与 s 为备忘录



● 最短路径

● 计算关键

● 不同的子问题

● 迭代顺序

● 具体迭代例子

● 迭代实现

● 时间复杂度

● 实例

● 备忘录

● 标记函数

● 两种实现比较

● 小结

时间复杂度

根据伪码：行2, 3, 6都是 $O(n)$, 循环执行 $O(n^3)$, 内部为 $O(1)$

$$W(n) = O(n^3)$$

根据备忘录：估计每项工作量，求和. 子问题有 $O(n^2)$ 个，确定每个子问题的最少乘法次数需要对不同划分位置比较(第9行)，需要 $O(n)$ 时间.

$$W(n) = O(n^3)$$

追踪解工作量 $O(n)$, 总工作量 $O(n^3)$



● 最短路径

● 计算关键

● 不同的子问题

● 迭代顺序

● 具体迭代例子

● 迭代实现

● 时间复杂度

● 实例

● 备忘录

● 标记函数

● 两种实现比较

● 小结

实例

输入: $P = \langle 30, 35, 15, 5, 10, 20 \rangle$

$$n = 5$$

矩阵链: $A_1 A_2 A_3 A_4 A_5$.

$A_1: 30 \times 35$, $A_2: 35 \times 15$, $A_3: 15 \times 5$, $A_4: 5 \times 10$, $A_5: 10 \times 20$

备忘录: 存储所有子问题的最小乘法次数及得到这个值的划分位置.



最短路径

计算关键

不同的子问题

迭代顺序

具体迭代例子

迭代实现

时间复杂度

实例

备忘录

标记函数

两种实现比较

小结

备忘录 $m[i, j]$ $P = \langle 30, 35, 15, 5, 10, 20 \rangle$

$r=1$	$m[1,1]=0$	$m[2,2]=0$	$m[3,3]=0$	$m[4,4]=0$	$m[5,5]=0$
$r=2$	$m[1,2]=15750$	$m[2,3]=2625$	$m[3,4]=750$	$m[4,5]=1000$	
$r=3$	$m[1,3]=7875$	$m[2,4]=4375$	$m[3,5]=2500$		
$r=4$	$m[1,4]=9375$	$m[2,5]=7125$			
$r=5$	$m[1,5]=11875$				

递推方程:

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + P_{i-1}P_kP_j\} & i < j \end{cases}$$

$$m[2, 5] = \min\{0 + 2500 + 35 \times 15 \times 20, 2625 + 1000 + 35 \times 5 \times 20, 4375 + 0 + 35 \times 10 \times 20\} = 7125$$



● 最短路径

● 计算关键

● 不同的子问题

● 迭代顺序

● 具体迭代例子

● 迭代实现

● 时间复杂度

● 实例

● 备忘录

● 标记函数

● 两种实现比较

● 小结

标记函数 $s[i, j]$

$r=2$	$s[1,2]=1$	$s[2,3]=2$	$s[3,4]=3$	$s[4,5]=4$	
$r=3$	$s[1,3]=1$	$s[2,4]=3$	$s[3,5]=3$		
$r=4$	$s[1,4]=3$	$s[2,5]=3$			
$r=5$	$s[1,5]=3$				

解的追踪: $s[1, 5] = 3 \Rightarrow (A_1 A_2 A_3)(A_4 A_5)$
 $s[1, 3] = 1 \Rightarrow A_1(A_2 A_3)$

输出

计算顺序: $(A_1(A_2 A_3))(A_4 A_5)$

最少的乘法次数: $m[1, 5] = 11875$



● 最短路径

● 计算关键

● 不同的子问题

● 迭代顺序

● 具体迭代例子

● 迭代实现

● 时间复杂度

● 实例

● 备忘录

● 标记函数

● 两种实现比较

● 小结

两种实现的比较

递归实现：时间复杂性高，空间较小

迭代实现：时间复杂性低，空间消耗多

原因：递归实现子问题多次重复计算，子问题计算数呈指数增长.迭代实现每个子问题只计算一次.

动态规划时间复杂度：

备忘录各项计算量之和 + 追踪解工作量

通常追踪工作量不超过计算工作量，是问题规模的多项式函数

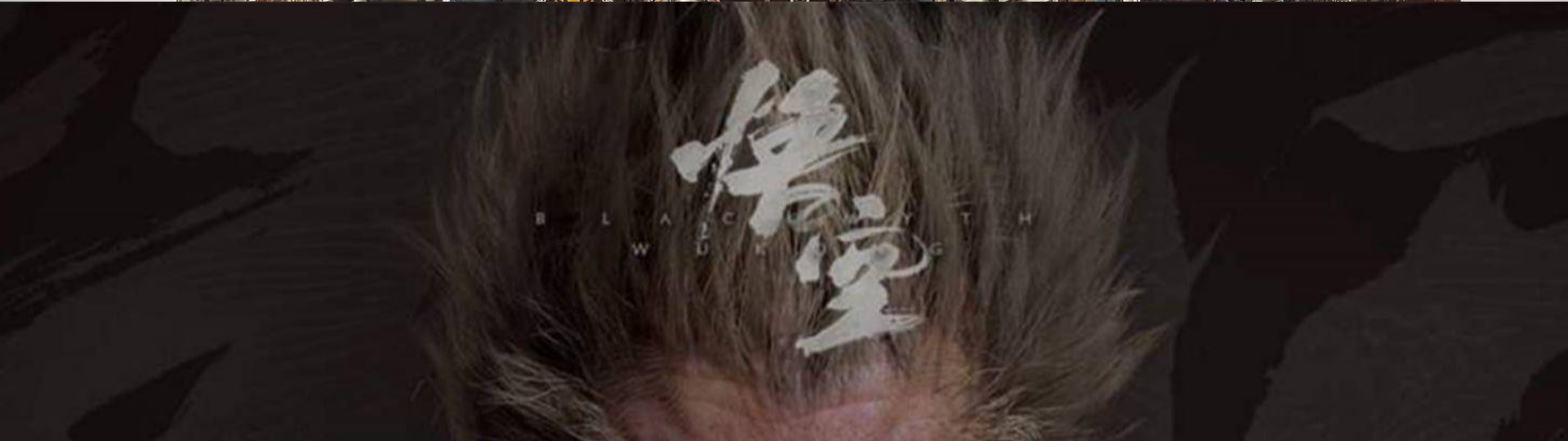


小结---动态规划算法的要素

- **划分子问题**，确定子问题边界，将问题求解转变成多步判断的过程.
- **定义优化函数**，以该函数极大（或极小）值作为依据，确定是否**满足优化原则**
- 列优化函数的**递推方程**和边界条件
- 自底向上计算，设计**备忘录**（表格）
- 考虑是否需要设立**标记函数**
- 用递推方程或备忘录估计**时间复杂度**



OVERWATCH





OVERWATCH





OVERWATCH



OVERWATCH





一、动态规划算法的例子

最短路径算法

二、动态规划的设计要素

递归实现

迭代实现

三、动态规划实例

投资问题

背包问题

最长公共子序列





● 投资问题

● 问题定义

● 实例

● 子问题界定

● 递推方程

● 实例计算1

● 实例计算2

● 备忘录和解

● 备忘录

● 时间复杂度

● 小结

投资问题的建模

问题： m 元钱， n 项投资， $f_i(x)$:将 x 元投入第 i 个项目的效益.求使得总效益最大的投资方案.

建模：

问题的解是向量 $\langle x_1, x_2, x_3, \dots, x_n \rangle$,

x_i 是投给项目 i 的钱数, $i = 1, 2, \dots, n$.

目标函数 $\max\{f_1(x_1) + f_2(x_2) + \dots + f_n(x_n)\}$

约束条件 $x_1 + x_2 + \dots + x_n = m, x_i \in \mathbb{N}$



● 投资问题

实例

实例：5万元钱，4个项目，效益函数如下表所示

x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24

● 问题定义

● 实例

● 子问题界定

● 递推方程

● 实例计算1

● 实例计算2

● 备忘录和解

● 备忘录

● 时间复杂度

● 小结



● 投资问题

● 问题定义

● 实例

● 子问题界定

● 递推方程

● 实例计算1

● 实例计算2

● 备忘录和解

● 备忘录

● 时间复杂度

● 小结

子问题界定和计算顺序

子问题界定：由参数 k 和 x 界定

k ：考虑对项目 $1, 2 \dots k$ 的投资

x ：投资总钱数不超过 x



这两个参数与矩阵链相乘问题的参数有什么区别？

原始输入： $k = n, x = m$

子问题计算顺序：

$k = 1, 2, \dots, n$

对于给定的 $k, x = 1, 2, \dots, m$



投资问题

- 问题定义
- 实例
- 子问题界定
- 递推方程
- 实例计算1
- 实例计算2
- 备忘录和解
- 备忘录
- 时间复杂度
- 小结

优化函数的递推方程

设 $F_k(x)$: 表示 x 元钱投给前 k 个项目的最大效益

多步判断: 假设知道 p 元钱 ($p < x$) 投给前 $k-1$ 个项目的最大效益 $F_{k-1}(p)$, 确定 x 元钱投给前 k 个项目的分配方案

递推方程和边界条件

$$F_k(x) = \max_{0 \leq x_k \leq x} \{f_k(x_k) + F_{k-1}(x - x_k)\} \quad k > 1$$

$$F_1(x) = f_1(x)$$



投资问题

问题定义

实例

子问题界定

递推方程

实例计算1

实例计算2

备忘录和解

备忘录

时间复杂度

小结

实例的计算 $k=1$

x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	12	13	31
3	13	16	30	33
4	14	21	41	50
5	15	26	43	61

$k=1$ 为初值

$$F_1(1) = 11, F_1(2) = 12, F_1(3) = 13,$$

$$F_1(4) = 14, F_1(5) = 15$$

● 投资问题

● 问题定义

● 实例

● 子问题界定

● 递推方程

● 实例计算1

● 实例计算2

● 备忘录和解

● 备忘录

● 时间复杂度

● 小结

实例的计算 $k=2$

方案: (其他, 项目2) : $(0, 1), \underline{(1, 0)}$

$$F_2(1) = \max\{f_2(1), \underline{f_1(1)}\} = 11$$

方案: $(0, 2), (1, 1), \underline{(2, 0)}$

$$F_2(2) = \max\{f_2(2), F_1(1) + f_2(1), \underline{F_1(2)}\} = 12$$

方案: $(0, 3), \underline{(1, 2)}, (2, 1), (3, 0)$

$$F_2(3) = \max\{f_2(3), \underline{F_1(1) + f_2(2)}, F_1(2) + f_2(1), F_1(3)\} = 16$$

类似计算

$$F_2(4) = 21, F_2(5) = 26$$

x	$f_1(x)$	$f_2(x)$
0	0	0
1	11	0
2	12	5
3	13	10
4	14	15
5	15	20



投资问题

问题定义

实例

子问题界定

递推方程

实例计算1

实例计算2

备忘录和解

备忘录

时间复杂度

小结

备忘录和解

$x_k(x)$ 标记函数

x	$F_1(x) \ x_1(x)$	$F_2(x) \ x_2(x)$	$F_3(x) \ x_3(x)$	$F_4(x) \ x_4(x)$
1	11 1	11 0	11 0	20 1
2	12 2	12 0	13 1	31 1
3	13 3	16 2	30 3	33 1
4	14 4	21 3	41 3	50 1
5	15 5	26 4	43 4	61 1

$$x_4(5) = 1 \Rightarrow x_4 = 1, x_3(5 - 1) = x_3(4)$$

$$x_3(4) = 3 \Rightarrow x_3 = 3, x_2(4 - 3) = x_2(1)$$

$$x_2(1) = 0 \Rightarrow x_2 = 0, x_1(1 - 0) = x_1(1)$$

$$x_2(1) = 1 \Rightarrow x_1 = 1$$

$$\text{解: } x_1 = 1, x_2 = 0, x_3 = 3, x_4 = 1, F_4(5) = 61$$



投资问题

问题定义

实例

子问题界定

递推方程

实例计算1

实例计算2

备忘录和解

备忘录

时间复杂度

小结

时间复杂度分析

备忘录表中有 m 行 n 列, 共计 mn 项

$$F_k(x) = \max_{0 \leq x_k \leq x} \{f_k(x_k) + F_{k-1}(x - x_k)\} \quad k > 1$$

$$F_1(x) = f_1(x)$$

对第 $F_k(x)$ 项 ($2 \leq k \leq n, 1 \leq x \leq m$)
要 $x + 1$ 次加法, x 次比较

加法次数

$$\sum_{k=2}^n \sum_{x=1}^m (x + 1) = \frac{1}{2} (n - 1) m (m + 3)$$

比较次数

$$\sum_{k=2}^n \sum_{x=1}^m x = \frac{1}{2} (n - 1) m (m + 1)$$

蛮力算法: 插板法

$$C(m + n - 1, m)$$

$$= \frac{(m + n - 1)!}{m! (n - 1)!}$$

蛮力算法的复杂度是多少?

$$W(n) = O(nm^2)$$



小结

投资问题的动态规划

- 用两个不同类型的参数界定子问题
- 列出优化函数的递推方程及初值
- 确定子问题计算顺序
- 根据备忘录中项的计算估计时间复杂度
- 时间复杂度 $O(nm^2)$



一、动态规划算法的例子

最短路径算法

二、动态规划的设计要素

递归实现

迭代实现

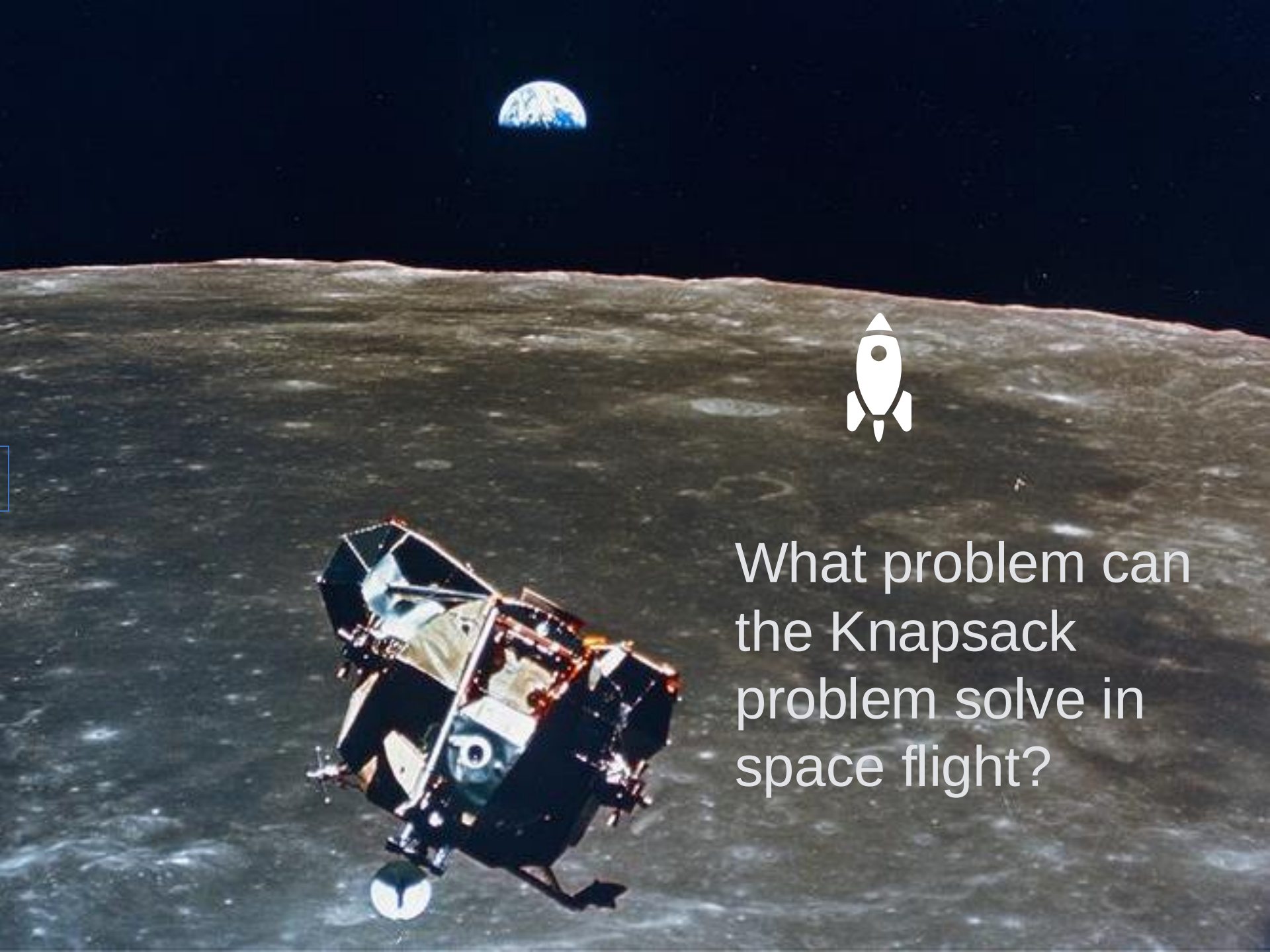
三、动态规划实例

投资问题

背包问题

最长公共子序列





What problem can
the Knapsack
problem solve in
space flight?



● 背包问题

● 问题定义

● 建模

● 子问题界定

● 递推方程

● 标记函数

● 实例

● 追踪解

● 追踪算法

● 时间复杂度

● 背包问题推广

● 小结

背包问题(Knapsack Problem)

问题：一个旅行者随身携带一个背包，可以放入背包的物品有 n 种，每种物品的重量和价值分别为 w_i, v_i .如果背包的最大重量限制是 b ，每种物品可以放多个，怎么选择放入背包的物品以使得背包的价值最大？不妨设上述 w_i, v_i, b 都是正整数。

实例：

$$n = 4, b = 10$$

$$v_1 = 1, v_2 = 3, v_3 = 5, v_4 = 9$$

$$w_1 = 2, w_2 = 3, w_3 = 4, w_4 = 7,$$



● 背包问题

● 问题定义

● 建模

● 子问题界定

● 递推方程

● 标记函数

● 实例

● 追踪解

● 追踪算法

● 时间复杂度

● 背包问题推广

● 小结

建模

解是 $\langle x_1, x_2, \dots, x_n \rangle$, 其中 x_j 是装入背包的第 j 种物品个数

目标函数

$$\max \sum_{j=1}^n v_j x_j$$

约束条件

$$\sum_{j=1}^n w_j x_j \leq b, \quad x_j \in \mathbb{N}$$

线性规划问题

由线性条件约束的线性函数取最大或最小的问题

整数规划问题 (另外的名称)

线性规划问题的变量 x_j 都是非负整数



● 背包问题

● 问题定义

● 建模

● 子问题界定

● 递推方程

● 标记函数

● 实例

● 追踪解

● 追踪算法

● 时间复杂度

● 背包问题推广

● 小结

子问题界定和计算顺序

子问题界定：由参数 k 和 y 界定

k ：考虑对物品 $1, 2, \dots, k$ 的选择

y ：背包总重量不超过 y

原始输入： $k = n, y = b$

子问题计算顺序：

$k = 1, 2, \dots, n$

对于给定的 $k, y = 1, 2, \dots, b$



● 背包问题

● 问题定义

● 建模

● 子问题界定

● 递推方程

● 标记函数

● 实例

● 追踪解

● 追踪算法

● 时间复杂度

● 背包问题推广

● 小结

优化函数的递推方程

$F_k(y)$: 装前 k 种物品, 总重不超过 y , 背包的最大价值

递推方程、边界条件、标记函数

$$F_k(y) = \max\{F_{k-1}(y), F_k(y - w_k) + v_k\}$$

$$F_0(y) = 0, \quad 0 \leq y \leq b, \quad F_k(0) = 0, \quad 0 \leq k \leq n$$

$$F_1(y) = \left\lfloor \frac{y}{w_1} \right\rfloor v_1, \quad F_k(y) = -\infty \quad y < 0$$

❓ 类似于投资问题, 如何写递推方程?
这两种写法由什么区别?

$$F_k(y) = \max_{0 \leq x_k \leq \left\lfloor \frac{y}{w_k} \right\rfloor} \{F_{k-1}(y - x_k w_k) + x_k v_k\}$$



● 背包问题

● 问题定义

● 建模

● 子问题界定

● 递推方程

● 标记函数

● 实例

● 追踪解

● 追踪算法

● 时间复杂度

● 背包问题推广

● 小结

标记函数

$i_k(y)$: 装前 k 种物品, 总重不超过 y , 背包达最大价值时装入物品的最大标号

$$i_k(y) = \begin{cases} i_{k-1}(y) & F_{k-1}(y) > F_k(y - w_k) + v_k \\ k & F_{k-1} \leq F_k(y - w_k) + v_k \end{cases}$$

$$i_1(y) = \begin{cases} 0 & y < w_1 \\ 1 & y \geq w_1 \end{cases}$$



● 背包问题

● 问题定义

● 建模

● 子问题界定

● 递推方程

● 标记函数

● 实例

● 追踪解

● 追踪算法

● 时间复杂度

● 背包问题推广

● 小结

实例

输入: $v_1 = 1, v_2 = 3, v_3 = 5, v_4 = 9$

$w_1 = 2, w_2 = 3, w_3 = 4, w_4 = 7,$

$b = 10$

$F_k(y)$ 的计算表如下:

$k \backslash y$	1	2	3	4	5	6	7	8	9	10
1	0	1	1	2	2	3	3	4	4	5
2	0	1	3	3	4	6	6	7	9	9
3	0	1	3	5	5	6	8	10	10	11
4	0	1	3	5	5	6	9	10	10	12



背包问题

问题定义

建模

子问题界定

递推方程

标记函数

实例

追踪解

追踪算法

时间复杂度

背包问题推广

小结

追踪解

$$i_k(y) = \begin{cases} i_{k-1}(y) & F_{k-1}(y) > F_k(y - w_k) + v_k \\ k & F_{k-1} \leq F_k(y - w_k) + v_k \end{cases}$$

$k \ y$	1	2	3	4	5	6	7	8	9	10
1	0	1	1	1	1	1	1	1	1	1
2	0	1	2	2	2	2	2	2	2	2
3	0	1	2	3	3	3	3	3	3	3
4	0	1	2	3	3	3	4	3	4	4

在上例中, 求得 $i_4(10)=4 \Rightarrow x_4 \geq 1$ (表示物品4被装入)

$$i_4(10 - w_4) = i_4(3) = 2 \Rightarrow x_2 \geq 1, x_4 = 1, x_3 = 0$$

$$i_2(3 - w_2) = i_2(0) = 0 \Rightarrow x_2 = 1, x_1 = 0$$

解 $x_1=0, x_2=1, x_3=0, x_4=1$, 价值12

● 背包问题

● 问题定义

● 建模

● 子问题界定

● 递推方程

● 标记函数

● 实例

● 追踪解

● 追踪算法

● 时间复杂度

● 背包问题推广

● 小结

追踪算法

输入: $i_k(y)$ 表, 其中 $k = 1, 2, \dots, n$, $y = 1, 2, \dots, b$

输出: $x_1, x_2, \dots, x_n$, n 种物品的装入量

1. for $j=1$ to n do

2. $x_j \leftarrow 0$

初始追踪
位置

3. $y \leftarrow b, j \leftarrow n$

4. $j \leftarrow i_j(y)$

5. $x_j \leftarrow 1$

6. $y \leftarrow y - w_j$

继续放
种物品

7. while $i_j(y) = j$ do

8. $y \leftarrow y - w_j$

9. $x_j \leftarrow x_j + 1$

10. if $i_j(y) \neq 0$ then goto 4

继续追下
一种



● 背包问题

● 问题定义

● 建模

● 子问题界定

● 递推方程

● 标记函数

● 实例

● 追踪解

● 追踪算法

● 时间复杂度

● 背包问题推广

● 小结

时间复杂度 $O(nb)$

根据公式

$$F_k(y) = \max\{F_{k-1}(y), F_k(y - w_k) + v_k\}$$

备忘录需计算 nb 项，每项常数时间，计算时间为 $O(nb)$ 。

伪多项式时间算法：时间为参数 b 和 n 的多项式，不是输入规模的多项式。

参数 b 是整数，表达 b 需要 $\log b$ 位，输入规模以 n 和 $\log b$ 为参数。



● 背包问题

● 问题定义

● 建模

● 子问题界定

● 递推方程

● 标记函数

● 实例

● 追踪解

● 追踪算法

● 时间复杂度

● 背包问题推广

● 小结

背包问题的推广

物品数受限背包：第 i 种物品最多用 n_i 个

0-1背包问题： $x_i = 0, 1, i = 1, 2, \dots, n$

多背包问题： m 个背包，背包 j 装入最大重量 B_j ， $j = 1, 2, \dots, m$. 在满足所有背包重量约束条件下使装入物品价值最大.

二维背包问题：每件物品有重量 w_i 和体积 t ，背包总重不超过 b ，体积不超过 V ，如何选择物品以得到最大价值.



小结

- 背包问题的子问题界定
- 列优化函数的递推方程和边界条件
- 自底向上计算，设计备忘录（表格）
- 标记函数的设立和追踪算法
- 背包问题的推广及应用



一、动态规划算法的例子

最短路径算法

二、动态规划的设计要素

递归实现

迭代实现

三、动态规划实例

投资问题

背包问题

最长公共子序列





子序列问题

问题定义

最长公共子序列

蛮力算法

子问题界定

依赖关系

递推方程

标记函数

伪码

追踪解

标记函数实例

时空复杂度

小结

子序列

设序列 X, Z :

$$X = \langle x_1, x_2, \dots, x_m \rangle$$

$$Z = \langle z_1, z_2, \dots, z_k \rangle$$

若存在 X 的元素构成的严格递增序列
使得

$$z_j = x_{i_j}, j = 1, 2, \dots, k$$

则称 Z 是 X 的**子序列**

X 与 Y 的公共子序列 Z : Z 是 X 和 Y 的子序列
子序列的长度: 子序列的元素个数



子序列问题

问题定义

最长公共子序列

蛮力算法

子问题界定

依赖关系

递推方程

标记函数

伪码

追踪解

标记函数实例

时空复杂度

小结

最长公共子序列

问题：给定序列

$$X = \langle x_1, x_2, \dots, x_m \rangle,$$

$$Y = \langle y_1, y_2, \dots, y_n \rangle$$

求 X 和 Y 的最长公共子序列

实例

X : **A** **B** **C** **B** **D** **A** **B**

Y : **B** **D** **C** **A** **B** **A**

一个最长公共子序列: **B C B A, 长度为4**



子序列问题

问题定义

最长公共子序列

蛮力算法

子问题界定

依赖关系

递推方程

标记函数

伪码

追踪解

标记函数实例

时空复杂度

小结

蛮力算法

不妨设 $m \leq n, |X| = m, |Y| = n$

算法：依次检查 X 的每个子序列在 Y 中是否出现

时间复杂度：

每个子序列 $O(n)$ 时间

X 有 2^m 个子序列

最坏情况下时间复杂度： $O(n2^m)$



子序列问题

问题定义

最长公共子序列

蛮力算法

子问题界定

依赖关系

递推方程

标记函数

伪码

追踪解

标记函数实例

时空复杂度

小结

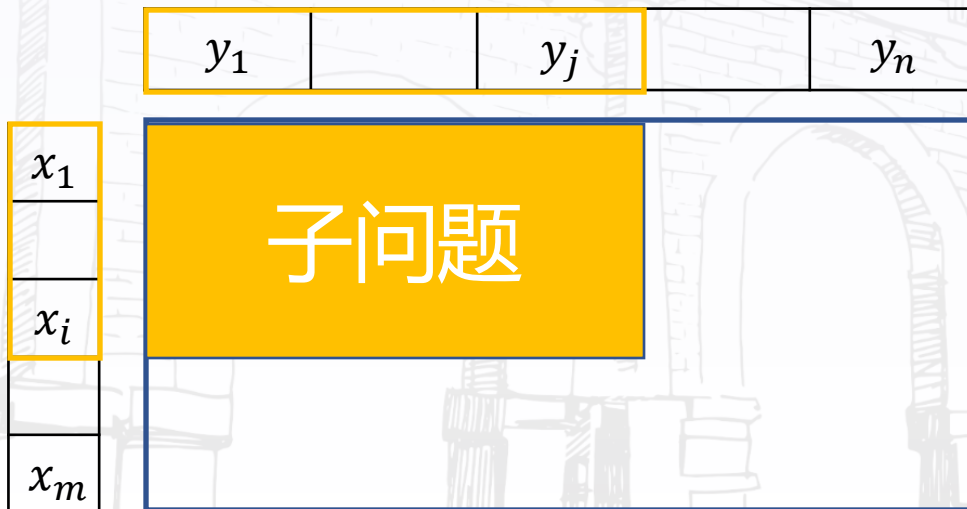
子问题的界定

参数*i*和*j*界定子问题算法

*X*的终止位置是*i*, *Y*的终止位置是*j*

$$X = \langle x_1, x_2, \dots, x_i \rangle,$$

$$Y = \langle y_1, y_2, \dots, y_j \rangle$$





子序列问题

问题定义

最长公共子序列

蛮力算法

子问题界定

依赖关系

递推方程

标记函数

伪码

追踪解

标记函数实例

时空复杂度

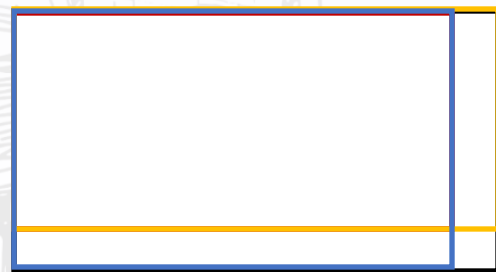
小结

子问题间的依赖关系

设 $X = \langle x_1, x_2, \dots, x_m \rangle$, $Y = \langle y_1, y_2, \dots, y_n \rangle$, $Z = \langle z_1, z_2, \dots, z_k \rangle$, Z 为 X 和 Y 的 LCS, 那么

- (1) 若 $x_m = y_n \Rightarrow z_k = x_m = y_n$, 且 Z_{k-1} 是 X_{m-1} 与 Y_{n-1} 的 LCS;
- (2) 若 $x_m \neq y_n$, $z_k \neq x_m \Rightarrow Z$ 是 X_{m-1} 与 Y 的 LCS;
- (3) 若 $x_m \neq y_n$, $z_k \neq y_n \Rightarrow Z$ 是 X 与 Y_{n-1} 的 LCS.

满足优化原则和子问题重叠性





子序列问题

问题定义

最长公共子序列

蛮力算法

子问题界定

依赖关系

递推方程

标记函数

伪码

追踪解

标记函数实例

时空复杂度

小结

优化函数的递推方程

令 X 与 Y 的子序列

$$X_i = \langle x_1, x_2, \dots, x_i \rangle, \quad Y_j = \langle y_1, y_2, \dots, y_j \rangle$$

$C[i, j]$: X_i 与 Y_j 的 LCS 的长度

递推方程

$$C[i, j] = \begin{cases} 0 & \text{若 } i = 0 \text{ 或 } j = 0 \\ C[i-1, j-1] + 1 & \text{若 } i, j > 0, x_i = y_j \\ \max\{C[i, j-1], C[i-1, j]\} & \text{若 } i, j > 0, x_i \neq y_j \end{cases}$$

子序列问题

问题定义

最长公共子序列

蛮力算法

子问题界定

依赖关系

递推方程

标记函数

伪码

追踪解

标记函数实例

时空复杂度

小结

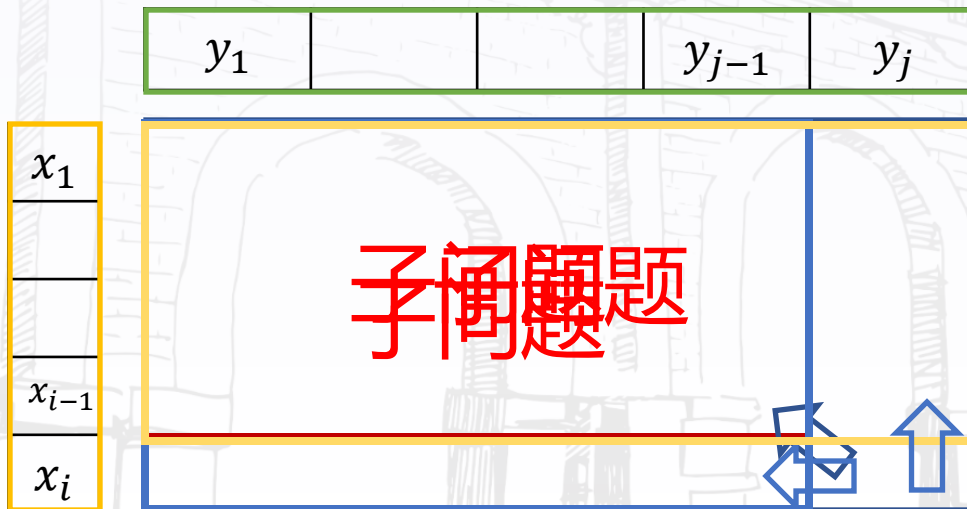
标记函数

标记函数: $B[i,j]$, 值为 $\nwarrow$ 、 $\leftarrow$ 、 $\uparrow$

$$C[i,j] = C[i-1,j-1] + 1 : \nwarrow$$

$$C[i,j] = C[i,j-1] : \leftarrow$$

$$C[i,j] = C[i-1,j] : \uparrow$$





子序列问题

问题定义

最长公共子序列

蛮力算法

子问题界定

依赖关系

递推方程

标记函数

伪码

追踪解

标记函数实例

时空复杂度

小结

伪码

算法 $LCS(X, Y, m, n)$

初始值

```
1. for  $i \leftarrow 1$  to  $m$  do  $C[i, 0] \leftarrow 0$  //行1-2边界情况
2. for  $i \leftarrow 1$  to  $n$  do  $C[0, i] \leftarrow 0$ 
3. for  $i \leftarrow 1$  to  $m$  do
4.   for  $j \leftarrow 1$  to  $n$  do //子问题  $X_i, Y_i$ 
5.     if  $X[i] = Y[j]$ 
6.       then  $C[i, j] \leftarrow C[i-1, j-1] + 1$ 
7.        $B[i, j] \leftarrow \nwarrow$ 
8.     else if  $C[i-1, j] \geq C[i, j-1]$ 
9.       then  $C[i, j] \leftarrow C[i-1, j]$ 
10.       $B[i, j] \leftarrow \uparrow$ 
11.     else  $C[i, j] \leftarrow C[i, j-1]$ 
12.       $B[i, j] \leftarrow \leftarrow$ 
```



子序列问题

问题定义

最长公共子序列

蛮力算法

子问题界定

依赖关系

递推方程

标记函数

伪码

追踪解

标记函数实例

时空复杂度

小结

追踪解

算法 Structure Sequence (B, i, j)

输入: $B[i, j]$

输出: X 与 Y 的最长公共子序列

1. if $i = 0$ or $j = 0$ then return //一个序列为空
2. if $B[i, j] = \text{“}\nwarrow\text{”}$
3. then 输出 $X[i]$
4. Structure Sequence ($B, i-1, j-1$)
5. else if $B[i, j] = \text{“}\uparrow\text{”}$
6. then Structure Sequence ($B, i-1, j$)
7. else Structure Sequence ($B, i, j-1$)



子序列问题

问题定义

最长公共子序列

蛮力算法

子问题界定

依赖关系

递推方程

标记函数

伪码

追踪解

标记函数实例

时空复杂度

小结

标记函数实例

输入： $X = \langle A, B, C, B, D, A, B \rangle$, $Y = \langle B, D, C, A, B, A \rangle$,

标记函数：

	1	2	3	4	5	6
1	$B[1,1] = \uparrow$	$B[1,2] = \uparrow$	$B[1,3] = \uparrow$	$B[1,4] = \nearrow$	$B[1,5] = \leftarrow$	$B[1,6] = \nwarrow$
2	$B[2,1] = \nwarrow$	$B[2,2] = \leftarrow$	$B[2,3] = \leftarrow$	$B[2,4] = \uparrow$	$B[2,5] = \nwarrow$	$B[2,6] = \leftarrow$
3	$B[3,1] = \uparrow$	$B[3,2] = \uparrow$	$B[3,3] = \nwarrow$	$B[3,4] = \leftarrow$	$B[3,5] = \uparrow$	$B[3,6] = \uparrow$
4	$B[4,1] = \nwarrow$	$B[4,2] = \uparrow$	$B[4,3] = \uparrow$	$B[4,4] = \uparrow$	$B[4,5] = \nwarrow$	$B[4,6] = \leftarrow$
5	$B[5,1] = \uparrow$	$B[5,2] = \nwarrow$	$B[5,3] = \uparrow$	$B[5,4] = \uparrow$	$B[5,5] = \uparrow$	$B[5,6] = \uparrow$
6	$B[6,1] = \uparrow$	$B[6,2] = \uparrow$	$B[6,3] = \uparrow$	$B[6,4] = \nwarrow$	$B[6,5] = \uparrow$	$B[6,6] = \nwarrow$
7	$B[7,1] = \nwarrow$	$B[7,2] = \uparrow$	$B[7,3] = \uparrow$	$B[7,4] = \uparrow$	$B[7,5] = \nwarrow$	$B[7,6] = \uparrow$

解： $X[2], X[3], X[4], X[6]$, 即 B, C, B, A



子序列问题

问题定义

最长公共子序列

蛮力算法

子问题界定

依赖关系

递推方程

标记函数

伪码

追踪解

标记函数实例

时空复杂度

小结

算法时空复杂度

计算优化函数和标记函数：

赋初值，为 $O(m) + O(n)$

计算优化、标记函数迭代 $\Theta(mn)$ 次，
循环体内常数次运算，时间为 $\Theta(mn)$

构造解：

每步缩小X或Y的长度，时间 $O(m + n)$

算法时间复杂度： $\Theta(mn)$

空间复杂度： $\Theta(mn)$



小结

- 最长公共子序列问题建模
- 子问题边界的界定
- 递推方程及初值，优化原则判定
- 伪码
- 标记函数与解的追踪
- 时空复杂度

作业：币值最大化问题

随机给定 n 个硬币，其面值均为正整数 $c_1, c_2, \dots, c_n$ ，硬币的面值可以两两相同。请问如何选择硬币，使得在其原始位置互不相邻的条件下，所选的硬币总金额最大。

样例：

5, 1, 2, 10, 6, 2

输出：

17